

Gov Hub — Requirements Document

Prepared for: Bridgit DAO

Version: 1.0

Date: August 10, 2026

Repository: [Bridgit-DAO/interface-gov-hub](#)

Status: Living document — reflects production system and GOV-HUB-3 evolution plan

Document control

Field	Value
Owner	Bridgit DAO / Meta-Layer stewardship
Audience	Bridgit DAO engineering, product, and governance contributors
Canonical architecture	<code>GOV-HUB-3.md</code> (v3.0, March 2026)
Production URLs	hub.themetalayer.org , govhub.live , rfc.themetalayer.org
Development URLs	dev.hub.themetalayer.org , dev.govhub.live

Related documents

Document	Location	Purpose
GOV-HUB-3 Architecture	<code>GOV-HUB-3.md</code>	Authoritative build plan and gap analysis
Design Decisions	<code>DESIGN_DECISIONS.md</code>	Governance philosophy and terminology
Soft Launch Canvas	<code>docs/META_LAYER_SOFT_LAUNCH_CANVAS.md</code>	Product narrative and artifact lifecycle UI
Access Model + OpenAPI	<code>docs/GOV_HUB_ACCESS_SSOT_AND_OPENAPI.md</code>	Join policies, visibility, API contract
Artifact Specification	<code>artifact_specification.md</code>	Typed knowledge objects and relations
Dev → Main Workflow	<code>docs/DEV-TO-MAIN-WORKFLOW.md</code>	Branching, deploy, and promote policy
Layers/Workgroups/Guilds Status	<code>PROJECTS_WORKGROUPS_GUILDS_STATUS.md</code>	Feature completion tracker

1. Executive summary

Gov Hub (Interface Governance Hub) is the coordination layer for the Meta-Layer ecosystem. It is where ideas become proposals, proposals become decisions, and

decisions become implementation. The platform evolved from an IETF Datatracker fork and is now a distinct Flask application operated by Bridgit DAO.

Gov Hub is **not a greenfield project**. It is a production system serving live governance workflows today, with an approved architectural evolution (GOV-HUB-3) toward an artifact-driven knowledge and decision model.

Strategic intent for Bridgit DAO

Bridgit DAO stewards Gov Hub as infrastructure for:

1. **Open participation** — anyone can contribute drafts, comment, join guilds, and engage in layer-scoped governance without creating a layer first.
2. **Structured deliberation** — workgroups assess rough consensus; layers provide long-lived stewardship containers.
3. **Verifiable decisions** — voting with eligibility snapshots, audit trails, and optional Bitcoin Ordinals provenance.
4. **Cross-platform coordination** — integrations with Canopi (discussion), Desirable Properties (editorial rails), Metaweb (badge wallet), and knowledge-graph agents.

Guiding principle

Evolve, don't replace. All new work must preserve working production features. Migrations are incremental, tested on dev, and backed up before production promotion.

2. Vision and product goals

2.1 Product vision

Headline: Build decisions, not just discussions.

Subtext: A coordination layer where ideas become proposals, and proposals become reality.

2.2 Core user journeys

Journey	Description	Success criteria
Contribute	User shares an idea, insight, or ML-Draft	Contribution is visible, attributable, and discussable
Review	Peers support, oppose, comment, or add evidence	Clear signal of community sentiment without silencing dissent
Deliberate	Workgroup or layer coordinates refinement	Rough consensus documented; chairs/coordinators facilitate, not dictate
Decide	Layer-scoped vote reaches quorum and threshold	Eligibility snapshotted; outcome recorded immutably
Implement	Approved work moves to implementation status	Traceable lineage from idea → decision → action
Immortalize	Optional on-chain provenance via Ordinals	Draft or badge inscribed with verifiable Bitcoin anchor

2.3 Governance philosophy

From `DESIGN_DECISIONS.md`:

- **Layers are optional stewardship containers**, not prerequisites for participation.
- Most users submit to existing layers (especially Meta-Layer); few create new layers.
- **Workgroups** assess rough consensus but do **not** exercise unilateral authority.
- **Guilds** enable cross-layer collaboration via invitation-only membership.
- Layer hierarchy is **flat** (no parent/child layers).
- Layer status is **descriptive, not evaluative** (proposed → active → stabilizing → maintaining → dormant → concluded → archived).

3. Stakeholders and roles

3.1 Platform stakeholders

Stakeholder	Interest
Bridgit DAO	Repository ownership, roadmap, elevated invite quotas for <code>@bridgit.io</code> / <code>@bridgitdao.com</code> / <code>@bridgitdao.io</code>
Meta-Layer layer stewards	Default governance home for ecosystem-wide work
Layer initiators and admins	Stewardship of individual layers
Workgroup coordinators and chairs	Task-focused deliberation within a layer
Guild initiators and admins	Cross-layer community formation
Contributors	Draft authors, commenters, voters, badge claimants
Integrators	Canopi, Desirable Properties, Metaweb, Hermes agents

3.2 System roles (authorization)

Site-level roles

Role	Capabilities
admin	Full site administration; bypasses most layer checks
editor	Site moderation staff; workgroup approval, scoped moderation
user	Default authenticated participant

Layer roles

Role	Capabilities
Initiator	Layer owner; full layer administration
Layer admin	Assigned administration; config, members, invitations
Layer member	Active membership; voting eligibility, quest participation
Anonymous	Public listing metadata only (subject to <code>listing_visibility</code>)

Workgroup roles

Role	Capabilities
Coordinator	Workgroup creator/lead
Chair / co-lead	Approved leadership positions
Member	Active participant; can invite (with policy)
Site staff override	Admins/editors can manage per policy

Guild roles

Role	Capabilities
Initiator	Auto-admin on creation
Admin	Membership and configuration
Member	Participation after invitation acceptance

4. Functional requirements

Requirements are grouped by module. Status indicators:

-  **Shipped** — in production
-  **Partial** — implemented but incomplete or migrating
-  **Planned** — approved in GOV-HUB-3, not yet built

4.1 Identity and authentication

ID	Requirement	Status	Notes
AUTH-01	Users authenticate via Web3Auth (Google, Twitter/X, email passwordless, wallet connect)	✓	JWT <code>idToken</code> verified server-side
AUTH-02	Flask session cookies for browser clients	✓	Documented in OpenAPI
AUTH-03	Optional TOTP MFA	✓	<code>routes/mfa.py</code>
AUTH-04	Social OAuth account linking (Google, GitHub, Discord, Twitter)	✓	Linking only; not primary login
AUTH-05	User profiles with display name, directory listing, public ID	✓	UUID migration in progress
AUTH-06	IdentityAnchor abstraction for cross-layer identity	📋	User model serves initially
AUTH-07	WalletBinding (on-demand EVM/BTC Taproot)	📋	Phase 1+

4.2 Layers (formerly Projects)

ID	Requirement	Status	Notes
LAY-01	Create layer with admin approval workflow	✓	Prevents spam
LAY-02	Layer status lifecycle (proposed → archived)	✓	Descriptive statuses with <code>status_reason</code>
LAY-03	Layer membership: join, leave, admin assignment	✓	
LAY-04	Layer vanity subdomains (<code>[slug].hub.themetalayer.org</code>)	✓	Nginx wildcard TLS
LAY-05	Listing visibility: <code>public</code> <code>private</code>	✓	
LAY-06	Join policy: <code>open</code> <code>by_invitation</code> <code>nft_gated</code>	↺	Enforcement shipped; join-policy UI incomplete
LAY-07	Layer activity feed from EventLog	✓	
LAY-08	Layer connections (typed connectors between layers)	✓	
LAY-09	Layer programs and document naming prefixes	✓	
LAY-10	Layer invitations and email campaigns	✓	
LAY-11	Full rename Project → Layer in code and schema	↺	Approved migration; UUID PKs

4.3 Workgroups

ID	Requirement	Status	Notes
WG-01	Layer-scoped workgroup creation	✓	Creator becomes coordinator
WG-02	Editor/admin approval before workgroup is active	✓	Only approved WGs in submission forms
WG-03	Membership: join, leave, member requests	✓	
WG-04	Leadership: coordinator, chair, co-lead nominations	✓	Email notifications to layer admins
WG-05	Member invitations with optional AI-assisted drafting	✓	<code>workgroup_invite_ai.py</code>
WG-06	Workgroup chat/messages	✓	EventLog <code>workgroup_message_posted</code>
WG-07	Document assignment to workgroups	✓	Required for RFC promotion
WG-08	No limit on workgroups per layer	✓	Design decision

4.4 Guilds

ID	Requirement	Status	Notes
GLD-01	Cross-layer guild creation (no admin approval)	✓	Initiator becomes admin
GLD-02	Invitation-only membership (<code>GuildInvitation</code> tokens, 7-day expiry)	✓	
GLD-03	Guild-layer links, artifact links, quest links	✓	Partial OpenAPI coverage
GLD-04	Guild admin management	✓	

4.5 Documents and drafts (IETF heritage)

ID	Requirement	Status	Notes
DOC-01	ML-Draft submission and revision workflow	✓	
DOC-02	Comments, patches, revision diffs	✓	
DOC-03	Document follow subscriptions with email notifications	✓	Links to Canopi where configured
DOC-04	Workgroup association required for RFC promotion	✓	
DOC-05	Ordinal-sourced drafts with Bitcoin provenance	✓	<code>sourceType</code> , <code>ordinalId</code> , etc.
DOC-06	DP Proposals: sentence-level reader edits on Desirable Properties chapters	✓	Accept/decline by WG coordinator, chair, layer admin
DOC-07	Every draft/RFC associates with a layer	✓	Most use existing Meta-Layer

4.6 Voting and ballots

ID	Requirement	Status	Notes
VOT-01	Layer-scoped votes referencing submissions	✓	Migrating to artifact references
VOT-02	Vote lifecycle: scheduled → active → closed	✓	
VOT-03	Eligibility snapshot at activation from active layer members	✓	VoteEligibilitySnapshot
VOT-04	Ballot choices; election mode with candidates and seats	✓	
VOT-05	Automated vote tick (systemd timer)	✓	gov-hub-vote-tick.service
VOT-06	Vote references artifact_id instead of submission_id	↺	GOV-HUB-3 migration

4.7 Roles, claims, and badges

ID	Requirement	Status	Notes
ROL-01	Layer-scoped roles organized in clusters	✓	
ROL-02	Role claims with evidence and optional approval workflow	✓	
ROL-03	Badge issuance with approval workflow	✓	
ROL-04	Badge custody modes: custodial or user-wallet	✓	
ROL-05	Role image voting	✓	
ROL-06	BRC333 badge registry and admin bindings	✓	
ROL-07	Badge Keeper Role with PEARL reflection path	📋	Phase 2-3
ROL-08	Triad model (role-anchored, max 3; distinct from guild)	📋	May map to workgroup with type=triad

4.8 Artifact system (GOV-HUB-3)

ID	Requirement	Status	Notes
ART-01	Central Artifact model (Proposal, Evidence, Insight, etc.)		Base model exists; submission migration incomplete
ART-02	Artifact status lifecycle: draft → under_review → vote_scheduled → vote_open → approved → implemented		Soft-launch UI demonstrates full flow
ART-03	Artifact actions: Support, Oppose, Comment, Add Evidence		Wired when <code>SOFT_LAUNCH_WIRED_ARTIFACT_ID</code> set
ART-04	Typed ArtifactRelation: supports, opposes, builds_on, etc.		No <code>contradicts</code> on artifacts (use <code>opposes</code>)
ART-05	Bridges: claim-centric links to external URLs/evidence		<code>cites</code> , <code>contradicted_by</code> , etc.
ART-06	Collections (curated artifact groupings)		
ART-07	Quests and Monuments		Layer-scoped
ART-08	Signal artifact (daily civic clock)		Design only

4.9 Bitcoin and Ordinals

ID	Requirement	Status	Notes
BTC-01	Read inscription content from ordinals.com		Markdown rendering
BTC-02	Unisat inscription wizard (Phase 1)		<code>docs/UNISAT_INSCRIBE_TAB_SPEC.md</code>
BTC-03	Custodial Taproot wallets for badge provenance signing		BIP86 via embit
BTC-04	Tier pricing (phone-based)		<code>docs/TIER_PRICING.md</code>
BTC-05	Standalone <code>/inscribe/</code> page		

4.10 Notifications and activity

ID	Requirement	Status	Notes
NTF-01	In-app notifications (<code>UserNotification</code>)		
NTF-02	Email follow subscriptions (<code>UserEventSubscription</code>)		Resend transactional email
NTF-03	EventLog append-only governance audit trail		<code>docs/EVENT_TYPES.md</code>
NTF-04	Layer activity feeds		

4.11 Platform operations

ID	Requirement	Status	Notes
OPS-01	Product rollout feature flags (layers, docs, votes, artifacts, etc.)	✓	/admin/product-rollout/
OPS-02	Platform-wide invitations with Bridgit org elevated quotas	✓	
OPS-03	Referral attribution	✓	
OPS-04	Support tickets with Hermes AI triage	✓	Production branch
OPS-05	Waitlists	✓	
OPS-06	i18n shell and Civic Mason localization	↔	docs/SHELL_I18N.md
OPS-07	Civic Mason brick-placement engagement	✓	

4.12 Admin and moderation

ID	Requirement	Status	Notes
ADM-01	Site admin dashboard	✓	
ADM-02	Layer creation approval	✓	
ADM-03	Workgroup creation approval	✓	
ADM-04	BRC333 badges admin	✓	
ADM-05	Product rollout admin	✓	
ADM-06	All user-facing dialogs use GhDialog (no native alert / confirm)	✓	Workspace rule

5. Integration requirements

5.1 Canopi (discussion platform)

ID	Requirement	Direction	Auth
INT-CAN-01	Provision MetaCommunity when layer configured	Gov Hub → Canopi	Config URLs
INT-CAN-02	Mirror membership; document-follow emails link to Canopi	Gov Hub → Canopi	
INT-CAN-03	Workgroup membership check API	Canopi → Gov Hub	GOV_HUB_API_KEY Bearer
INT-CAN-04	Layer admin check API	Canopi → Gov Hub	Bearer
INT-CAN-05	Contribution webhooks	Canopi → Gov Hub	HMAC CANOPI_SIGNING_SECRET
INT-CAN-06	Custodial BTC provenance signing	Canopi → Gov Hub	Internal API

Canopi codebase references: `canopi/server/lib/govHub*.js`

5.2 Desirable Properties (editorial)

ID	Requirement	Notes
INT-DP-01	Gov Hub is editorial SSOT for DP book content	
INT-DP-02	Rails sync via GitHub Actions (govhub-rail-sync.yml)	desirable-properties repo
INT-DP-03	DP Challenge hub integration	routes/ dp_challenge_pages.py
INT-DP-04	DP proposal accept/decline authority aligned with workgroup roles	

5.3 Metaweb book

ID	Requirement	Notes
INT-MW-01	Badge wallet API	routes/metaweb.py
INT-MW-02	Pioneer layer gating	Stripe server integration
INT-MW-03	Action-status health checks	Used in deploy verification

5.4 Knowledge graph agents

ID	Requirement	Notes
INT-KG-01	Hermes Gov Hub agent	neo4j-knowledge-graph/src/hermes/govhub-*.js
INT-KG-02	DP memory graph proposal/contribution sync	dp-memory-graph/src/sync/govhub-*.js

5.5 External services

Service	Purpose
Web3Auth	Primary authentication
Resend	Transactional email
Unisat API	Inscription creation
ordinals.com	Inscription read
GitHub	Source control; DP rail sync dispatch

6. API requirements

6.1 Contract status

OpenAPI v0.1.0 at `openapi/openapi.yaml` is a **partial SSOT**. Documented paths include guilds, quests, and artifact guild-links. Many endpoints remain undocumented and should be treated as unstable until added to OpenAPI.

Contract reference: docs/GOV_HUB_ACCESS_SSOT_AND_OPENAPI.md

6.2 Authentication modes

Mode	Use case
Flask session cookie	Browser clients
Bearer Web3Auth token	DP internal APIs
Authorization: Bearer \$GOV_HUB_API_KEY	Canopi/Hermes internal
X-Canopi-Signature HMAC	Canopi webhooks

6.3 API surface (by area)

Prefix	Areas
/api/layers/	CRUD, members, join/leave, activity, guilds, invitations
/api (workgroups)	Members, messages, nominations, invite-AI
/api/guilds/	CRUD, invites, layer/artifact/quest links
/api (artifacts)	Artifacts, quests, monuments, relations, support/opposition
/api (votes)	Create, ballot, close, candidates
/api (roles)	Clusters, roles, claims, badges
/api/bridges/	Bridge CRUD, sessions, inscribe
/api/auth/web3auth	Login
/api/internal/canopi/	Membership/admin checks
/api/soft-launch/	Demo fixtures and lifecycle
/doc/	Draft reader, comments, patches (HTML + JSON)

6.4 Bridgit DAO deliverable: API documentation expansion

Requirement API-DOC-01: Prioritize OpenAPI documentation for layers, workgroups, votes, and auth endpoints before external integrator onboarding.

7. Non-functional requirements

7.1 Performance and scale

ID	Requirement	Current state
NFR-01	Support concurrent governance activity for Meta-Layer and multiple active layers	SQLite WAL mode; production-proven at current scale
NFR-02	Vote tick processes scheduled activations/closures within 1 minute of due time	systemd timer
NFR-03	Plan Postgres migration when team scale or concurrency demands	 Trigger: team scale or Alembic adoption

7.2 Reliability and operations

ID	Requirement	Current state
NFR-04	Hourly database backup	Cron on VPS
NFR-05	Deploy via <code>deploy.py</code> / rollback via <code>rollback.py</code>	Scripts exist; roadmap notes reliability gaps
NFR-06	Pre-promote checklist on dev before merge to <code>main</code>	<code>docs/DEV-T0-MAIN-WORKFLOW.md</code>
NFR-07	Hotfix on <code>main</code> requires same-day backport to <code>development</code>	Policy
NFR-08	Upload store path must be configurable (not hardcoded)	 Known gap in <code>app.py</code>

7.3 Security

ID	Requirement
NFR-09	No secrets in repository or Jau memories
NFR-10	Web3Auth JWT verification on all auth endpoints
NFR-11	Internal APIs require API key or HMAC signature
NFR-12	MFA available for high-value accounts
NFR-13	Per-service secret isolation (shared <code>OPENAI_API_KEY</code> across 9 services is a known estate gap)

7.4 Accessibility and UX

ID	Requirement
NFR-14	All success/error/confirm UI via <code>GhDialog</code> styled dialogs
NFR-15	Bootstrap 5 + custom <code>govhub-design.css</code> design system
NFR-16	i18n support for shell and Civic Mason (expanding)

7.5 Testing

ID	Requirement
NFR-17	<code>python3 test_core_features.py</code> passes before promote
NFR-18	pytest suite for changed modules
NFR-19	Manual verification on <code>dev.hub.themetalayer.org</code> before production merge

8. Technical architecture

8.1 Stack summary

Layer	Technology
Backend	Flask + SQLAlchemy
Database	SQLite (dev: <code>instance_dev/datatracker_dev.db</code> , prod: <code>instance/datatracker.db</code>)
Frontend (new)	Vue 3 + Vite + Pinia
Frontend (legacy)	jQuery + Parcel + Bootstrap 5
Auth	Web3Auth + Flask sessions
Email	Resend
BTC	embit (BIP86 Taproot), cryptography
OAuth linking	flask-dance
Python	3.9.18 (pyenv on VPS)
Migrations	Idempotent <code>migrate_*</code> in <code>migrations/__init__.py</code>

Note: README still references upstream Django/PostgreSQL/Docker. The **live Gov Hub runtime** is Flask (`app.py`), not Django.

8.2 Repository layout

Path	Branch	Role
<code>/home/ubuntu/gov-hub-dev</code>	<code>development</code>	Integration and testing (port 8001)
<code>/home/ubuntu/gov-hub-prod</code>	<code>main</code>	Live production (port 8000)

8.3 Deployment topology

Service	Unit	Port
Production app	<code>datatracker.service</code>	8000
Development app	<code>datatracker-dev.service</code>	8001
Vote tick (dev)	<code>gov-hub-vote-tick.service</code>	—
Vote tick (prod)	<code>gov-hub-vote-tick-prod.service</code>	—

Reverse proxy: nginx with wildcard TLS for layer vanity hosts.

Estate registry: `meta-console/registry.yaml` (project `gov-hub`).

8.4 Code organization (target state)

GOV-HUB-3 Phase 0 extracted the monolithic `ietf_data_viewer_simple.py` into:

- `models/` — domain models
- `routes/` — Flask blueprints

- `services/` — business logic
- `migrations/` — schema migrations

Further modular extraction continues per roadmap.

9. Roadmap and migration plan

9.1 Approved migrations

Migration	Scope	Policy
Project → Layer	Model, table, FKs, routes	Clean break; no backward-compat layer
UUID primary keys	User, Layer, Submission, Vote, etc.	Clean break; <code>CHAR(36)</code> in SQLite
Submission → Artifact subtype	Vote references, lifecycle UI	Incremental
Vote → Artifact reference	<code>artifact_id</code> replaces <code>submission_id</code>	Incremental

9.2 Migration safety rules

1. Never run destructive migrations on production without verified backup.
2. Always test on `gov-hub-dev` before promoting to `gov-hub-prod`.
3. New modules are additive; existing routes stay functional during transition.
4. Schema changes require migration script and rollback path.
5. EventLog is additive infrastructure; it does not replace existing tables.

9.3 Phase overview (GOV-HUB-3)

Phase	Focus	Key deliverables
0	Foundation	EventLog, modular extraction, ArtifactRelation
1	Artifact core	Artifact model, IdentityAnchor (or defer), Triad
2	Governance depth	Badge Keeper, Quests, Bridges
3	Reflection & monuments	PEARL, Monuments, Signal worker
Ongoing	Operations	OpenAPI expansion, Postgres decision, deploy hardening

9.4 Known gaps (August 2026)

Gap	Priority	Owner action
development ↔ main branch divergence	High	Resolve per docs/SYNC.md before external onboarding
OpenAPI incomplete	High	Document layers, workgroups, votes, auth
Join-policy UI incomplete	Medium	Complete access control matrix
Upload path hardcoded	Medium	Make configurable via env/config
SQLite → Postgres	Low (until scale)	Decision document when triggered
Signal artifact	Low	Design complete; implementation TBD

10. Operational requirements for Bridgit DAO

10.1 Branching and release policy

```
development → test on dev → merge to main → restart datatracker.service
```

- **All feature work** lands on `development`.
- **main is production.** No separate `production` branch.
- **Hotfixes** on `main` allowed when live is broken; backport to `development` same day.
- **Never duplicate fixes** on both branches.

10.2 Environment verification

```
# Dev health check (401 without auth is OK; 404 means wrong deploy)
curl -s -o /dev/null -w "%{http_code}\n" -X POST https://dev.govhub.live/api/metaweb/
action-status \
  -H 'Content-Type: application/json' -d '{"checks":[]}'
```

10.3 Pre-promote checklist (mandatory)

- [] `git status` clean; branch pushed to `origin/development`
- [] `systemctl --user restart datatracker-dev.service` — no traceback
- [] `python3 test_core_features.py` passes
- [] Relevant pytest modules pass
- [] Manual smoke test on dev URLs
- [] Database backup verified before prod restart

10.4 Monitoring and incident response

Resource	Location
Estate OPEN_ITEMS	meta-console/OPEN_ITEMS.md
Connectivity outage runbook	docs/GOVHUB_CONNECTIVITY_OUTAGE.md
Deployment checklist	docs/DEPLOYMENT-CHECKLIST.md
SSL/vanity docs	docs/GOVHUB_HUB_LAYER_VANITY.md , docs/GOVHUB_LIVE_SSL.md

11. Acceptance criteria (Bridgit DAO onboarding)

Bridgit DAO team onboarding is complete when:

1. **Access** — Team members have GitHub write access to `Bridgit-DAO/interface-gov-hub` and dev environment SSH/VPS access as agreed.
2. **Local/dev fluency** — Team can run dev app, execute test suite, and verify on `dev.hub.themetalayer.org`.
3. **Workflow adherence** — All changes follow `development` → test → `main` promote policy.
4. **Architecture alignment** — Team has read `GOV-HUB-3.md` and agrees to "evolve, don't replace."
5. **Branch sync** — `development` and `main` divergence resolved per `SYNC.md`.
6. **API contract** — Agreed priority list for OpenAPI expansion documented in GitHub issues.
7. **Decision log** — Postgres migration trigger criteria and artifact migration sequencing agreed in writing.

12. Open decisions for Bridgit DAO

#	Decision	Options	Recommendation
1	Artifact migration scope	Full GOV-HUB-3 vs maintain submission-centric votes	Incremental: artifact UI + parallel submission support until vote migration complete
2	Database strategy	Stay on SQLite vs plan Postgres	SQLite until concurrency/team scale triggers; document trigger criteria
3	Triad implementation	New model vs workgroup <code>type=triad</code>	Workgroup extension unless distinct semantics required
4	External API stability	Document-as-is vs versioned <code>/api/v1/</code>	Versioned prefix when OpenAPI coverage reaches critical mass
5	Deploy automation investment	Fix <code>deploy.py</code> reliability vs GitHub Actions CI/CD	Parallel: harden scripts short-term; CI/CD medium-term

13. Glossary

Term	Definition
Layer	Primary organizing entity for governance (formerly Project)
Workgroup	Task-focused group within a layer; assesses rough consensus
Guild	Cross-layer collaboration group; invitation-only
ML-Draft	Meta-Layer draft document (IETF-inspired)
Artifact	Typed knowledge object (proposal, evidence, insight, etc.)
Bridge	Claim-centric link between artifact and external evidence
EventLog	Append-only governance audit trail
Ordinal	Bitcoin inscription used for provenance
PEARL	Reflection artifact with badge overlay (planned)
Coordinator	Workgroup lead (formerly "Chair" in some contexts)
Initiator	Creator of a layer or guild (formerly "Submitted by")

14. Feature flags reference

Product rollout toggles (`services/product_rollout.py`):

`layers`, `docs`, `roles`, `workgroups`, `guilds`, `badges`, `waitlists`, `immortalize`, `votes`, `artifacts`, `quests`, `bridges`, `soft_launch`, `patches`, and others.

Admin UI: `/admin/product-rollout/`

15. Appendix: URL map

Environment	Apex domains	Layer vanity	Port
Production	hub.themetalayer.org, govhub.live, rfc.themetalayer.org, themetalayer.org	<code>[slug].hub.themetalayer.org</code>	8000
Development	dev.hub.themetalayer.org, dev.govhub.live	<code>[slug].dev.hub.themetalayer.org</code>	8001

16. Appendix: ecosystem projects

Project	Relationship
canopi	Discussion layer; membership sync
desirable-properties	Book rails sync from Gov Hub
metaweb-book	Badge wallet, pioneer gating
BRC333	Ordinals/badge tooling
neo4j-knowledge-graph	Hermes Gov Hub agent
dp-memory-graph	Proposal/contribution sync
meta-console	Estate monitoring and registry

This document synthesizes production codebase state, GOV-HUB-3 architecture, and operational runbooks as of August 10, 2026. Validate live behavior against dev URLs before treating any feature as externally stable.